Arkaraj Mukherjee
Homework 11

```r
library(tidyverse);


## - Attaching core tidyverse packages -------------------- tidyverse 2.0.0
-
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## - Conflicts ------------------------------ tidyverse_conflicts() -
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force
all conflicts to become errors


set.seed(123);
```

1. (a) Since $H_1 : m > m_0$, large values of the sign statistic $S = \#\{i : X_i > m_0\}$ favor the alternative. Under H_0, we have $S \sim \text{Binomial}(n, \frac{1}{2})$ so the rejection region at level α is

$$S \geq b^*, \quad \text{where } b^* = \min\left\{b : \sum_{k=b}^{n} \binom{n}{k} \left(\tfrac{1}{2}\right)^n \leq \alpha\right\}.$$

   (b) Since $H_1 : m < m_0$, small values of the sign statistic $S = \#\{i : X_i > m_0\}$ favor the alternative. Under H_0, we have $S \sim \text{Binomial}(n, \frac{1}{2})$, so the rejection region at level α is

$$S \leq b_*, \quad \text{where } b_* = \max\left\{b : \sum_{k=0}^{b} \binom{n}{k} \left(\tfrac{1}{2}\right)^n \leq \alpha\right\}.$$

2. (a) This is false, U in definition 1 is U_2 as in definition 3. Suppose we have $Y_{i_1} \leq \ldots \leq Y_{i_m}$ (i.e. Y_{i_k} is the sample with rank k) and let r_k be the rank of Y_{i_k} in the entire data set, then we clearly see that there are exactly $k-1$ Y_i's less than or equal to Y_{i_k} and thus exactly $r_k - 1 - (k-1) = r_k - k$ Y_i's less than or equal to X_{i_k}, thus we see that the presence of X_{i_k} contributes exaclty $r_k - k$ to U so we have $U = \sum_{k=m}^{n} r_k - k = W_2 - m(m+1)/2 = U_2$ where W_2 is as defined in definition 3 and so is U_2 and we also thus see that, (as per definition 3 for everything other than U) $P(W_2 \leq w) = P(U + m(m+1)/2 \leq w) = P(U \leq w - n(n+1)/2)$ so $C_1 = m(m+1)/2$. However, if X and Y were replaced in definition 1 then the claim would hold.

(b) $W_1 + W_2$ is the sum of all the ranks and is equal to $(n+m)(n+m+1)2$ and thus, $U_1 + U_2 = (n+m)(n+m+1)/2 - n(n+1)/2 - m(m+1)/2 = nm$. Thus, $P(U_2 \leq u_2) = P(nm - U_1 \leq u_2) = P(U_1 \geq nm - u_2)$ so we have $C_2 = nm$.

(c) This is false as per definition 3 (it is not mentioned which definition we have used so we go with 3 as otherwise the U defined in it would be redundant otherwise) We demonstrate this statistically below

```
experiment = function(num_sim,n,m,u_1,sampling_dist) {
u_2 = n*m - u_1
#calculating empirical probability of LHS under null hypothesis
#where X,Y have the same dist., say its standard normal
indicator_vec = replicate(num_sim,
{
X=sampling_dist(n)
Y=sampling_dist(m)
comb = c(X,Y)
ranks = rank(comb)
W_1 = sum(ranks[1:n])
W_2 = sum(ranks[(n+1):(n+m)])
U_1 = W_1 - n*(n+1)/2
U_2 = W_2 - m*(m+1)/2
U = min(U_1,U_2)
as.numeric(c(U_1 <= u_1, U_1 >= u_1, U <= min(u_1,u_2)))
})
#this is a 3x10^5 matrix
results = rowMeans(indicator_vec)
LHS = min(results[1:2])
RHS = results[3]
c(LHS,RHS)
}
#u_1 is atmost (m+1)+...+(m+n)-n(n+1)/2 = nm
v_1=experiment(10^5,30,40,467,rnorm)
v_2=experiment(10^5,10,20,100,rnorm)
# \(x) f(x,...) is basically treated as g(x):=f(x,...)
v_3=experiment(10^5,1200,300,170000,\(n) rpois(n,lambda=4))
temp = function(v) {v[1]/v[2]}
temp(v_1)

## [1] 0.5032572

temp(v_2)

## [1] 0.50813

temp(v_3)

## [1] 0.4958072
```

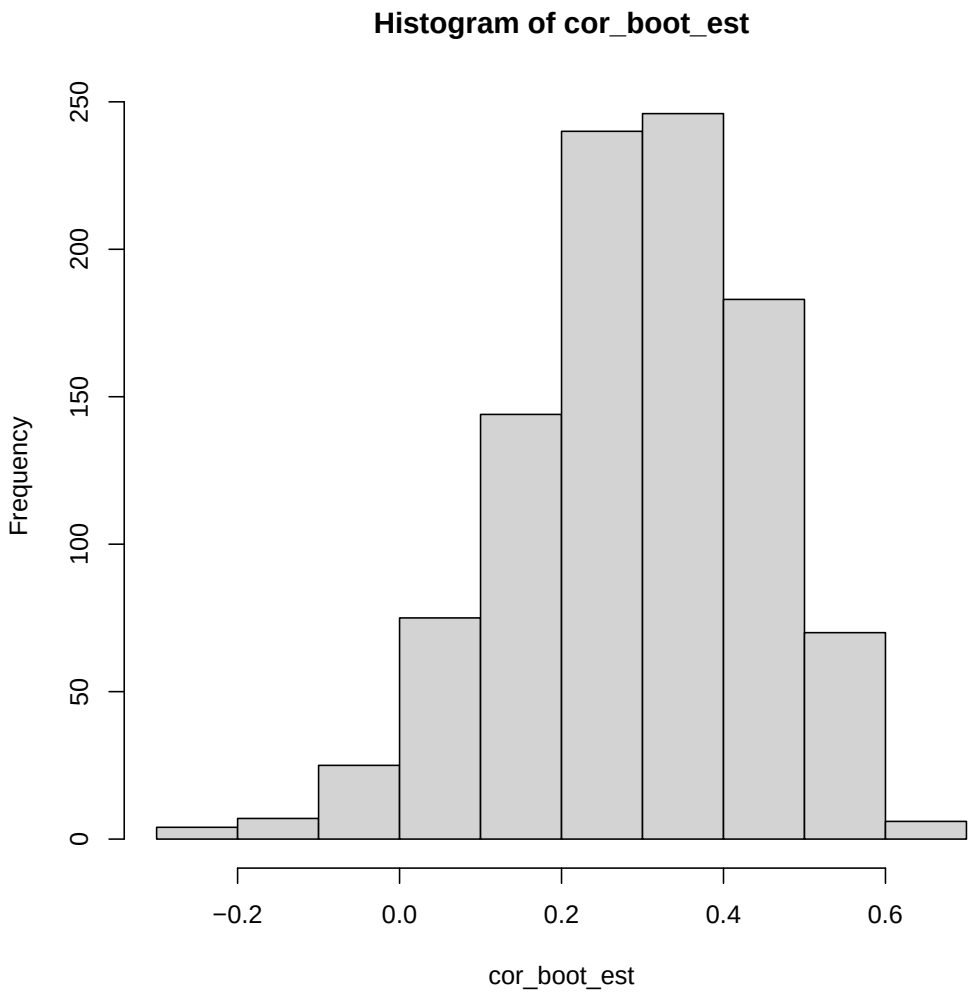
We can safely conclude that this is false, but the experiment suggests that $2 \times \texttt{LHS} = \texttt{RHS}$, this can be proved to be true when we sample from continuous distributions but it still fails for small discrete cases so theres no saving this.

3. (a)

```r
dat = read.csv("scores.csv")
head(dat)

##   Entrance UG
## 1      756 85
## 2      815 91
## 3      828 57
## 4      690 18
## 5      911 58
## 6      581 43

rho = cor(dat$Entrance,dat$UG)
rho

## [1] 0.2944079
```

(b) (and (c) as instructed)

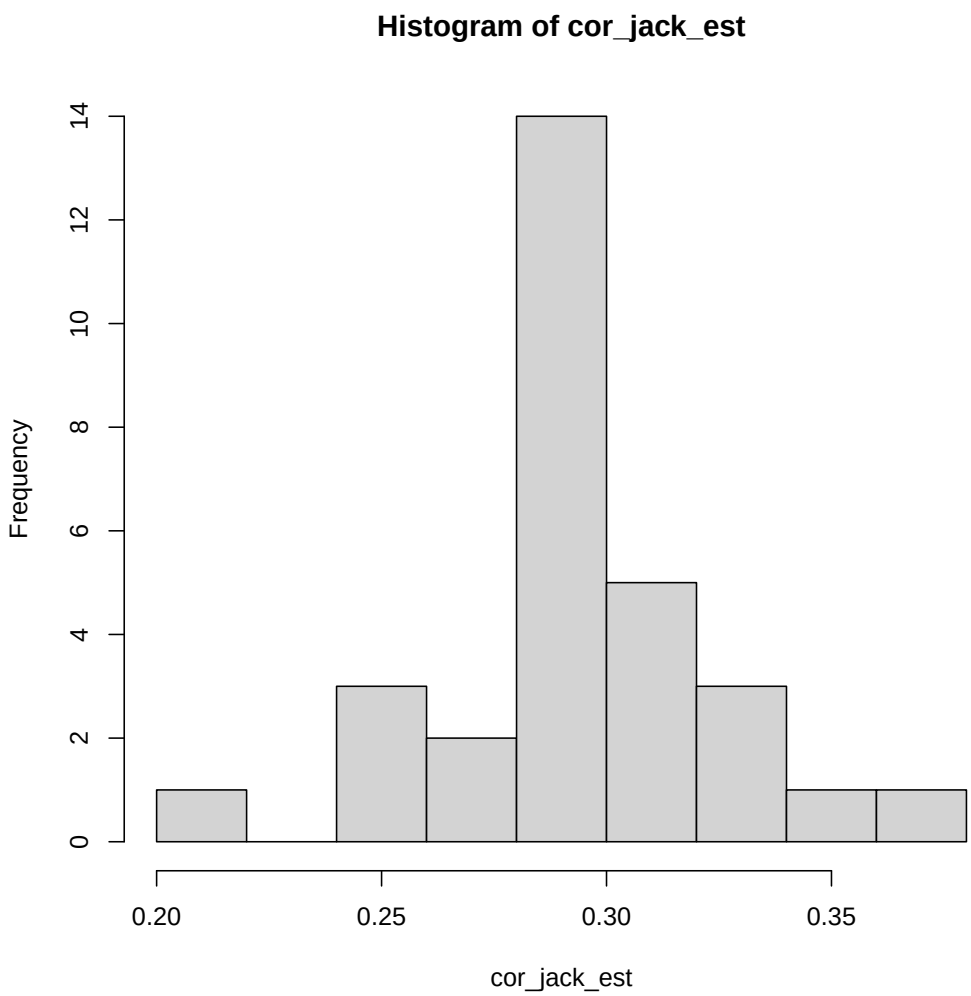
```r
# b
n = length(dat$UG)
cor_boot_est=replicate(1000,{
s = sample(1:n,n,replace=TRUE)
cor(dat$Entrance[s],dat$UG[s])
})
hist(cor_boot_est)
```




```
# c.(i)
cor_boot_bias = mean(cor_boot_est) - rho
cor_boot_bias

## [1] -0.001435587

cor_boot_se = sd(cor_boot_est)
cor_boot_se

## [1] 0.1507458

# c.(ii)
cor_boot_ci = as.vector(quantile(cor_boot_est, probs = c(0.025,0.975)))
cor_boot_ci

## [1] -0.02726172  0.54141480

# c.(iii)
```

```
cor_boot_pi = 2*c(rho,rho) - as.vector(quantile(cor_boot_est,
probs = c(0.975,0.025)))
cor_boot_pi

## [1] 0.04740092 0.61607745
```

(c)
```
cor_jack_est = vector(mode = "numeric", length = n)
for(i in 1:n) cor_jack_est[i] = cor(dat$Entrance[-i],dat$UG[-i])
hist(cor_jack_est)
```




```
cor_jack_bias = (n-1)*(mean(cor_jack_est)-rho)
cor_jack_bias

## [1] -0.003631554

cor_jack_se = ((n-1)/sqrt(n))*sd(cor_jack_est)
cor_jack_se
```

```
## [1] 0.155679
```